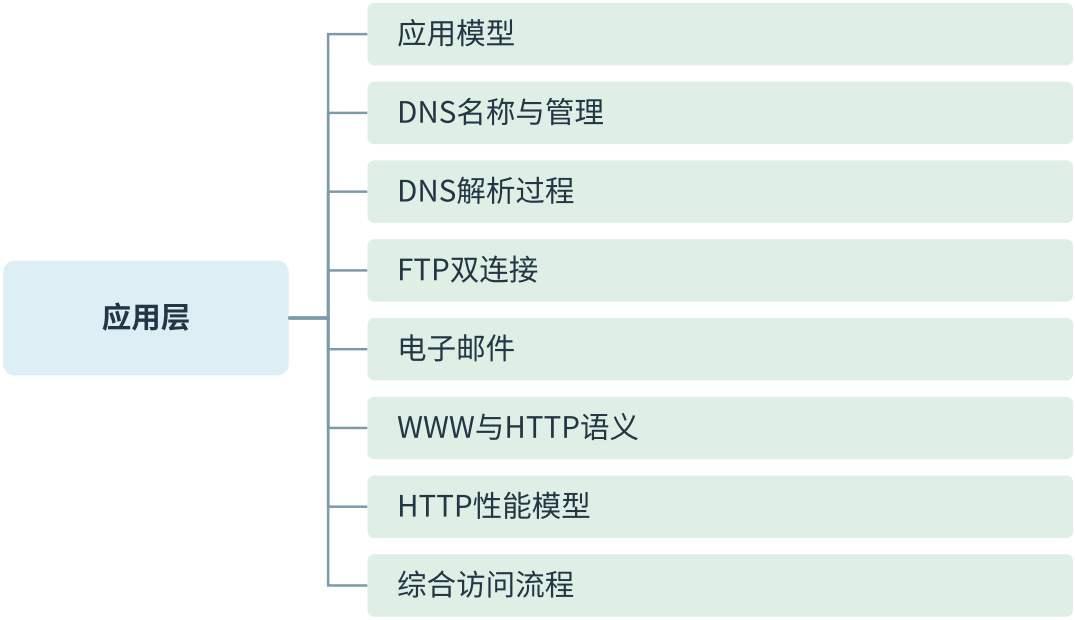
应用层 6

章节脉络



浏览器中的一个 URL 同时关联资源名称、服务器地址、传输连接和应用报文。计算访问时间时，要按依赖次序展开：先知道服务器地址，才能连接；先收到并解析 HTML，才能知道其中引用哪些资源。缓存命中、连接复用和流水请求改变的是这条依赖链中的具体步骤。

错项排除·角色由这次交互决定

一台机器可向 DNS 发查询，同时给其他客户提供 Web 服务。它在前一次交互中是客户，在后一次交互中是服务器；设备性能、是否长期在线以及机箱外观都不能替代应用角色的判断。

6.1 C/S 与 P2P：角色由谁发起服务请求决定

客户/服务器模型 C/S 中，服务器提供相对稳定的服务入口，客户主动发起请求。服务器可同时服务多个客户，客户不需要一直保持在线。这里“客户”与“服务器”是本次应用交互的角色，不是机器性能等级；同一主机可运行服务器，也可作为其他服务的客户。

P2P 让对等节点既能获取资源，也能向其他节点提供资源。它可以减少单一服务器的分发负担，但节点上下线、查找、安全和资源一致性更难管理。P2P 应用可以使用集中索引服务器辅助发现节点，不能因为存在一个中心组件，就断言所有数据必须经该中心转发。

以把大小为 F 的文件分发给 N 个接收者为例，纯 C/S 至少要求服务器上传 NF，因此时间不小于 NF/u_s；每个接收者还受自身下载速率限制。P2P 允许已收到数据的节点继续上传，能增加总上传能力，但仍至少受服务器上传、最慢接收者下载和全系统总上传量等条件限制。这个比较依赖可用上传率和持续参与等条件，不把“P2P 更快”写成不受网络约束的绝对结论。

文件分发先求不可突破的下界

设服务器上传率 u_s、最慢客户下载率 $d_{\min}$，客户上传率为 u_i，文件大小为 F，共有 N 个客户。纯 C/S 至少受

$$T_{CS} \geq \max\left(\frac{NF}{u_s}, \frac{F}{d_{\min}}\right)$$

限制；理想 P2P 还需服务器先贡献至少一份文件，且全系统总上传量要覆盖 NF，因此

$$T_{P2P} \geq \max\left(\frac{F}{u_s}, \frac{F}{d_{\min}}, \frac{NF}{u_s + \sum_i u_i}\right).$$

这些是持续参与、可用速率等理想条件下的必要下界，不能据此保证任意拓扑与实际调度都恰好达到。

推导只需守住数据守恒：纯 C/S 的 N 份副本都由服务器上传；P2P 则允许已收到数据的客户继续贡献上传，但所有副本的数据仍必须由系统某处发出。任何一个最慢下载者还必须收到完整 F，因此各资源下界应取最大值，不能相加成三个顺序阶段。

6.2 DNS 层次、服务器与区

DNS 把域名与资源记录关联。域名是从右向左逐级具体化的层次名称，完整域名可在末尾带根点。 `www.abc.com` 中 com 为顶级域，abc 是其下的一个域，www 是进一步的名称标签；不能只按点的个数断言每一级都由独立机器负责。

定义 6.2.1 (域与权威区)

域是名称空间中的子树，**区**是由某组权威服务器管理并提供权威数据的范围。一个域可以把某些子域委派出去，因此域与区不必相同。根服务器提供顶级域的委派信息，不存放全世界每台主机的最终地址；权威服务器回答其负责区内的记录。

常见记录有 A（IPv4 地址）、AAAA（IPv6 地址）、NS（权威服务器）、CNAME（规范名称别名）和 MX（邮件交换）。一个名称可以有多个地址，多个名称也可以指向同一个地址，域名、IP 与主机之间都不要求一一对应。〖6.2-001〗

域名书写要区分 “教材普通主机名规则” 与一般 DNS 记录名称。普通主机名标签常用字母、数字和连字符，大小写不敏感，较低层级写在左侧，顶级域在右侧。单标签最长 63 字节；DNS 线格式完整名称最长 255 字节，包含标签长度字节和末尾根标签。常见点分文本不带末尾根点时最多 253 字节，不能把线格式上限不加说明地当成任意显示字符串的字符上限。

顶级域可区分国家/地区类（如 cn）、通用类（如 com、org）与基础设施用途的 arpa 等。管理层级与实际服务器部署不是一一对应，反向名称解析的组织也不能简单写成把 IP 字符串交给普通 A 记录查询。

本地域名服务器即递归解析器，通常由组织或运营商等提供，位置不必与主机处在同一个以太网内。主机先查浏览器或系统缓存，再向配置的解析器发出请求；不能把本地解析器直接等同于根服务器。

最小反例·一个域可以跨多个区

abc.com 把子域 lab.abc.com 委派给另一组权威服务器后，lab 仍在 abc.com 名称子树内，却已不属于父区的直接权威数据范围。这个委派例子足以排除 “域和区永远完全相同” 的说法。

6.3 递归、迭代与缓存：谁又发了一次查询

不变量·每条查询都有发出者

主机向本地解析器的一条请求，不是本地解析器发出的请求。题目问某台服务器发出多少次时，应逐条画箭头并只数它发出的边；缓存命中可使远程次数归零，却不改变递归与迭代的定义。

递归查询要求被请求的服务器继续完成解析，最终返回答案或失败；迭代查询允许服务器返回下一步应询问的服务器信息，由原查询者继续询问。常见流程是主机向本地解析器请求递归服务，本地解析器依次对根、顶级、权威服务器进行迭代查询。

例如解析 `www.abc.com`：本地解析器先问根，取得 com 的服务器信息；再问 com，取得 abc.com 的权威服务器；最后问权威服务器取得 Web 地址。缓存中若已有最终 A/AAAA 记录，可不发远程查询；若只有某级委派，可以从该级继续。缓存受 TTL 有效期限制，不能无条件永久使用。

比较递归与迭代时，固定三件事：当前谁提出查询，收到的答复是最终答案还是下一站信息，由谁发下一条查询。递归把继续解析的工作交给被请求者，迭代让当前查询者沿委派继续。缓存命中只会改变需要走几步，并不改变这两种交互的定义；“用了缓存” 也不能直接推出请求来自根服务器还是本地解析器。

6.3.1 例 6-1　问的是本地服务器发出多少次查询

例 6.3.1 (配套例题)

配套例题。假设所有域名服务器均采用迭代查询方式进行域名解析。当 H4 访问规范域名为 `www.abc.xyz.com` 的网站时，域名服务器 201.1.1.1 完成该域名解析过程中，可能发出 DNS 查询的最少和最多次数分别是（　）。共用网络拓扑如下。

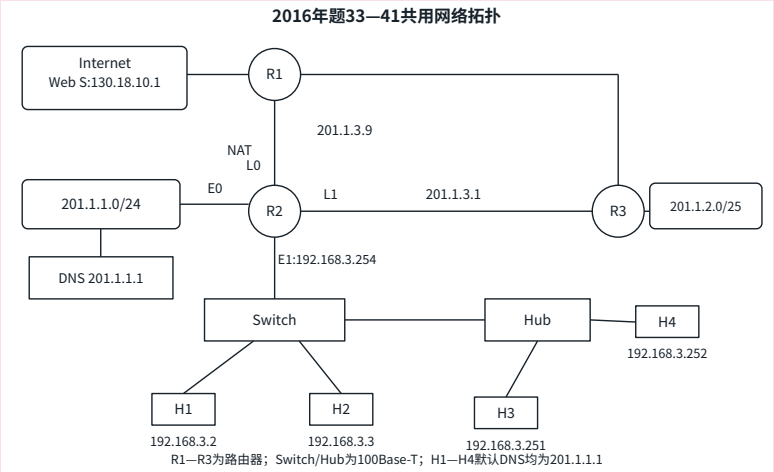


Figure 6.1: 2016 年共用拓扑；本题只使用 H4 及其默认 DNS 服务器 201.1.1.1

A. 0，3　　B. 1，3　　C. 0，4　　D. 1，4

解析。计数主体是 201.1.1.1，不是 H4。最终地址在有效缓存中时，本地服务器不需向外查询，最少 0 次；完全无可用缓存，且相应各级分别委派时，可依次询问根、com、xyz.com、abc.xyz.com，共 4 次，选 **C**。H4 向 201.1.1.1 的最初请求不是这台服务器 “发出” 的请求，不能加入。

“最多 4 次” 采用原题的规范域名及逐级委派模型，不计别名链、失败重试或负载均衡额外查询。若改问主机发出几条请求，或所有层级均以递归方式向下转交，通信主体与计数就要重新列出，而不是照抄 0、

4。

DNS 常用 UDP 端口 53，也支持 TCP，例如区域传送、较大响应或按协议退回 TCP 的场景。不能简单断言 DNS 只用 UDP，更不能将“DNS 也能用 TCP”误读为每次普通查询都必须先三次握手。

最小反例·数据端口不总是 20

FTP 被动模式由服务器通告一个监听数据端口，再由客户连接该端口。控制仍走服务器 21 端口，数据却可走另一个值，因此“服务器所有数据连接端口恒为 20”已被这个合法模式反驳。

6.4 FTP：控制命令与文件数据为什么分两条连接

FTP 运行在 TCP 上，控制连接用于登录、目录、传送命令与状态回复，服务器熟知端口为 21；数据连接用于文件或目录数据。在一次会话中，控制连接可持续存在，多次传送按需要建立、关闭数据连接，控制报文因此不必夹在文件字节流里，也能被识别。

6.4.1 例 6-2　传递 FTP 命令使用哪条连接

例 6.4.1 (配套例题)

配套例题。FTP 客户和服务器间传递 FTP 命令时，使用的连接是（　）。

A. 建立在 TCP 之上的控制连接

B. 建立在 TCP 之上的数据连接

C. 建立在 UDP 之上的控制连接

D. 建立在 UDP 之上的数据连接

解析。先由“FTP 命令”确定控制连接，再由 FTP 的传输协议确定 TCP，选 **A**。文件内容放在数据连接；登录服务器时，第一阶段建立的也是控制连接，不能等文件传送结束后才建立命令通道。〖6.3-005〗

主动模式由服务器向客户指定的地址和端口发起数据 TCP 连接，传统服务器数据源端口为 20；被动模式由服务器通告监听端口，再由客户主动连接该数据端口。不能说“所有 FTP 数据连接的服务器端口永远为 20”，也不能把主动/被动模式误解为控制连接的建立顺序。

教材常用“一个监听主进程，加多个会话控制/数据处理任务”解释 FTP 并发：监听 21 端口的入口接收新客户，每个会话维护自己的控制状态，具体文件传输再使用数据连接。实际服务器可用进程、线程或事件驱动实现，概念模型不要求每条连接都必须创建一个操作系统进程。

6.5 邮件系统、邮件格式与 MIME

邮件系统包括用户代理、邮件服务器和传送协议。用户代理负责编辑、显示和管理邮件，邮件服务器负责队列、邮箱及服务器间传送。一个“发送成功”的界面提示，可能只表示本机或提交服务器接受了邮件，不等于对方用户已经读取。

邮件由首部和正文组成，以空行分隔；首部中的 From、To、Subject 等表达用户可见信息，传输过程中 SMTP 还使用信封发件人与收件人控制实际投递。这两套地址可能不同，不能仅按邮件里显示的地址推断所有投递端点。

基础 SMTP 按 7 位 ASCII 文本传输，不能直接把任意二进制图像当作普通 7 位正文。MIME 定义内容类型、字符集、传送编码和多部分结构，使非 ASCII 文字与附件能表示为可传送内容。例如 Base64 将 3 字节转成 4 个 ASCII 字符，载荷大约膨胀到原来的 4/3，此外还有格式

开销。编码不是加密，收件人仍可按公开规则恢复原内容。现代扩展可协商更丰富的能力，作基础 SMTP 题时先确认题目采用哪一种范围。

不变量·读与发是不同应用动作

SMTP 负责提交和转送，POP3 负责读取邮箱。同一用户的一封邮件可能依次经过两种协议，不能因二者都处理邮件就互换。POP3 会话能读取多封，下载后是否删除还取决于相应操作和更新阶段。

6.6 SMTP 负责发送，POP3 负责读取

SMTP 是推送协议，用于用户代理提交邮件、以及邮件服务器之间转送邮件；服务器间常用 TCP 端口 25，用户提交还常见经认证的提交服务。POP3 是读取协议，用于用户代理从邮件服务器取邮件，基本服务使用 TCP 端口 110。协议名区分的是应用动作，不能因为都处理邮件就认为两者可以相互代替。

典型路径为：发送者用户代理通过 SMTP 提交 → 发送方邮件服务器按队列与 DNS 中的 MX 记录寻找收件域邮件服务器 → 服务器间通过 SMTP 转送 → 收件用户代理用 POP3 等读取。使用 Web 邮箱时，浏览器与 Web 服务还可能使用 HTTP，后端仍另有邮件传送机制。

POP3 依次经过授权、事务处理和更新等阶段。用户可列出、读取多封邮件，删除通常要等到会话的相应更新阶段才落实；不能把 “读取” 无条件等同于立即从服务器删除，也不能说一封邮件必新建一条 TCP 连接。

6.6.1 例 6-3　一次 POP3 连接可以读几封邮件

例 6.6.1 (配套例题)

配套例题。下列关于 POP3 协议的叙述中，正确的是（　）。

I. 支持用户代理从邮件服务器读取邮件

II. 支持用户代理向邮件服务器发送邮件

III. 支持邮件服务器之间发送与接收邮件

IV. 支持通过一条 TCP 连接收取多封邮件

A. 仅 I、IV　　B. 仅 II、III　　C. 仅 I、II、III　　D. 仅 I、III、IV

解析。 POP3 用于用户代理读取，I 正确；发送和服务器间转送由 SMTP 等机制承担，II、III 错误；事务阶段可在同一 TCP 连接内处理多封邮件，IV 正确，选 **A**。决定答案的是协议负责哪个方向，而不是 “都传邮件” 这个共同点。

6.7 WWW、URL 与资源位置

WWW 用超链接把资源组织起来，浏览器按 URL 定位和访问资源。URL 常包含方案、主机、可选端口、路径、查询及片段。例如 `http://www.abc.com:80/index.html?q=408#part1` 中，http 决定访问方案，主机名用于解析服务器地址，路径和查询参与 HTTP 请求定位，`#part1` 由浏览器用于文档内定位，不作为普通 HTTP 请求目标发送给服务器。

网页不是必然只有一份文件。HTML 可再引用图片、脚本、样式表等对象，浏览器通常要先收到 HTML，或在解析过程中逐步发现这些地址。它们可能来自同一服务器，也可能来自不同域名，需要另作 DNS 或连接判断。不能把 “同一个页面” 当成 “只需要一个网络往返”。

HTTP 是应用层协议，通常运行在 TCP 上；HTTPS 在 HTTP 通信中加入 TLS 等安全机制。本章时延题采用题目明确的 HTTP/1.0 或 HTTP/1.1 模型，不把 HTTP/2 多路复用或 HTTP/3 的 QUIC 机制混入旧

题。高版本存在，不会自动修改题干所限定的协议行为。

6.8 HTTP 请求、连接复用、缓存与时延

HTTP 请求行包含方法、请求目标、版本，后接首部、空行和可选消息体；响应包含状态行、首部及消息体。GET 用于获取资源，HEAD 获取相应首部信息，二者在协议语义上属于安全方法；POST 提交数据供资源处理，可能引起状态变化。判断题先按方法的规定语义作答；服务端的错误实现或附带日志等行为不能反过来改变这些方法的协议分类。状态码 200 表示请求成功，301/302 表示重定向，304 用于条件请求中的资源未改动，404 表示未找到等。

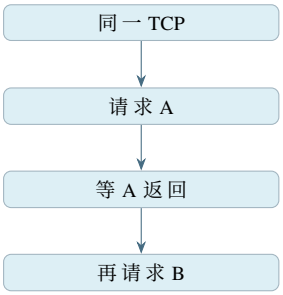
HTTP 被称为无状态协议，是指一次请求的处理不自动要求服务器记住上次会话状态。Cookie 可以让客户随请求携带标识，服务器据此关联状态；持久 TCP 连接则减少反复建立连接的开销。**无状态、Cookie 和连接是否持久是三个不同问题**，不能相互推出。

非持久 HTTP 通常每个对象使用一条 TCP 连接，题目忽略发送时延，一个小对象至少需要 1RTT 建立连接、1RTT 完成请求响应。若允许多个连接并行，可在发现对象后并行获取；若不允许并行，各对象就串行承担这两项。持久 HTTP 在同一连接上取多个对象，非流水方式仍按请求响应逐个等待；HTTP/1.1 流水线允许前一响应尚未完成就发后续请求，但尚不知道的资源地址无法凭空提前请求。

只有条件一致，比较才有可计算的结果：DNS 已完成、TCP 尚未建立，HTML 引用同站的 $m \geq 1$ 个小对象，必须先收到 HTML 才知道它们；不并行建连接，忽略报文传输、处理、拥塞与流量控制，所有往返均为同一个 RTT。

最小反例·持久连接仍可逐个等待

已建立一条 TCP 连接，先请求图片 A，等它完整返回后才请求 B，这是持久但非流水。两对象仍付两次请求响应等待，所以“用了持久连接就所有对象只要 1 RTT”的说法不成立。



持久连接仍可以不使用流水

模型	TCP 连接复用	对象请求能否重叠	从建立首连接到收齐的时间
非持久、串行	每个对象另建，共 $m+1$ 条	每次取完再建立下一条	$2(m+1)RTT$
持久、非流水	所有对象共用一条	HTML 及后续对象逐个等待	$(m+2)RTT$
持久、流水	所有对象共用一条	HTML 后，已知的 m 个请求可连续发出	$3RTT$

三行都包含首次 TCP 建立的 1RTT；若计时从已建连接上的 HTTP 请求发出开始，应各自去掉这一项。流水的 3RTT 仍含“握手—HTML—后续对象”三个依赖阶段；若对象较大、窗口不足或来自不同服务器，这张表的理想时间就不再直接适用。下面例题只改变其中的起点、并行条件或窗口约束。

HTTP 时延按依赖分段

先核对 DNS 是否完成、TCP 是否已建，再问 HTML 何时揭示资源地址；在后续资源阶段才区分是否并行、是否复用、是否流水。之后叠加题目明确给出的发送、拥塞窗口和处理开销。每次减少一个 RTT，都应说明是哪次握手或等待被免去。

例 6.8.1 (请求目标不包含片段标识)

访问 `http://www.abc.com/index.html?q=408#part1`，在题设 HTTP/1.1 模型中，请求可写为：

```
GET /index.html?q=408 HTTP/1.1
Host: www.abc.com
```

错项排除·流水不能猜未知资源

在 HTML 未返回且没有其他先验信息时，浏览器还不知道其中的图片 URL。允许流水只能把发现后的多个请求重叠，不能把 HTML 阶段也凭空抹去；这正是理想模型仍保留三个 RTT 阶段的原因。

路径与查询串交给服务器定位资源，`#part1` 留给浏览器在文档内定位；这段请求文本并未把 DNS、TCP 握手或 TLS 开销包含进去，计时仍须按题设另算。

6.8.1 例 6-4　非持久且不允许并行，要算上 HTML 本身

例 6.8.2 (配套例题)

配套例题。若浏览器不支持并行 TCP 连接，使用非持久的 HTTP/1.0 协议请求浏览 1 个 Web 页，该页中引用同一网站上的 7 个小图像文件，则从浏览器为传输 Web 页请求建立 TCP 连接开始，到接收完所有内容为止，所需要的 RTT 数至少是（　）。

A. 4　B. 9　C. 14　D. 16

解析。一共 8 个对象：HTML 与 7 张图片。每个对象单独建立 TCP 连接并请求响应，各 2RTT，且题目禁止并行，故总计 $8 \times 2 = 16RTT$，选 **D**。只计算 7 张图会得到 14，遗漏页面本身。题干计时从请求建立连接开始，不是从 HTTP 请求已经发出后开始。

6.8.2 例 6-5　流水线仍不能跨过资源发现依赖

例 6.8.3 (配套例题)

〖6.5-012〗假设主机通过 HTTP/1.1 流水线方式请求 Web 服务器上的 rfc.html，它引用同目录下 3 个 JPEG 小图像，只有收到 rfc.html 后才能发送对应图像请求。一次请求响应为 RTT，忽略其他时延，不考虑拥塞与流量控制。从发出 HTTP 请求报文开始，到收到全部内容，所耗时间是（　）。

A. 2RTT　B. 2.5RTT　C. 4RTT　D. 4.5RTT

解析。起点已经是 HTTP 请求发出，不再加 TCP 握手。第 1RTT 取 HTML，得到 3 个图片地址；3 个请求可流水发送，忽略传输时延时，再 1RTT 收到图片，共 2RTT，选 **A**。若题目不允许流水，应在同一连接逐对象等待，HTML 及 3 图共 4RTT。只有改变请求能否重叠这类条件，计算才会改变；不能把 “持久” 一词直接等同于 1RTT。

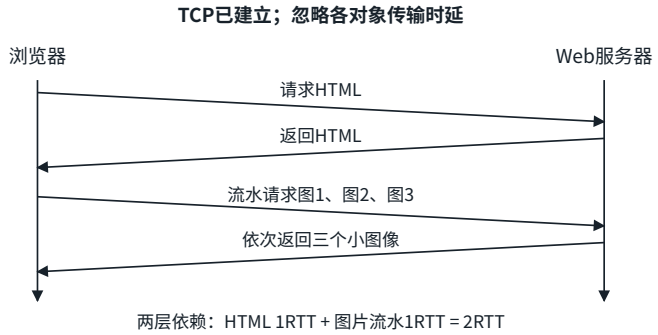


Figure 6.2: 连接已建立时，HTML 完成后才能发送三个图片请求；流水阶段可重叠

不变量·304 仍是一份网络响应

条件请求发到服务器后收到 304，说明发生了验证往返，只免去了完整实体内容。新鲜缓存直接命中则可能完全不发该请求。两种缓存行为都节省流量，却不具有相同的时延表达式。

6.8.3 例 6-6　DNS 与 Web 的时间由哪几项组成

例 6.8.4 (配套例题)

配套例题。假设下图网络中的本地域名服务器只提供递归查询服务，其他域名服务器均只提供迭代查询服务；局域网内主机访问 Internet 上各服务器的 RTT 均为 10 ms，忽略其他各种时延。若主机 H 通过超链接 `http://www.abc.com/index.html` 请求浏览纯文本 Web 页 index.html，从单击超链接到浏览器收到页面，所需的最短与最长时间分别是（　）。

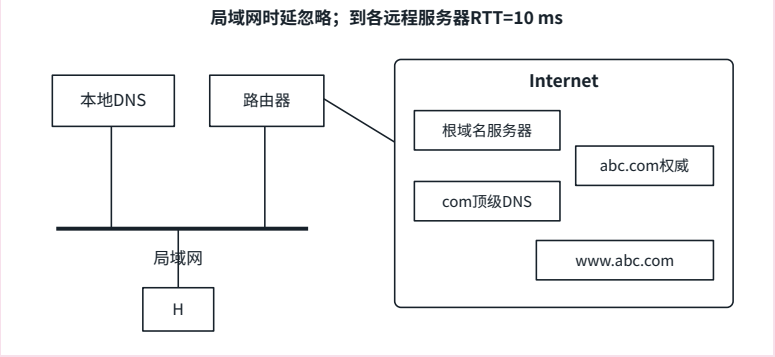


Figure 6.3: 主机、本地 DNS 与路由器共享局域网，远程服务器分别承担名称解析和 Web 响应

A. 10 ms，40 ms　　B. 10 ms，50 ms
C. 20 ms，40 ms　　D. 20 ms，50 ms

解析。采用题目首次取页面、尚未建立目标 TCP 连接的模型。有效 DNS 缓存命中时，不需要远程 DNS 往返，但 TCP 建立与 HTTP 请求响应各 1RTT，最短 20 ms。完全无所需缓存时，本地解析器依次询问根、com、abc.com 权威服务器，增加 3RTT；再取 Web 共 5RTT=50 ms，选 **D**。主机与本地 DNS 之间的局域网时延已忽略，不能额外加一次 10 ms。

如果题目再允许浏览器已缓存新鲜的 HTML 副本，可能无须访问服务器；若 TCP 已经建立，也会少一个握手往返。这些是改变原题模型的附加条件，不能在原题中任意引入后声称存在另一个更短答案。

6.8.4 例 6-7　收到 HTML 后，拥塞窗口还限制图片分几轮

例 6.8.5 (配套例题)

配套例题。假设主机 H 通过 HTTP/1.1 请求 Web 服务器 S 上的 news408.html，它引用同目录下 1 幅图像。HTML 文件大小为 1 MSS，图像 3 MSS，RTT=10 ms，忽略 HTTP 响应首部开销和 TCP 段传输时延。若 H 已完成域名解析，则从 H 请求与 S 建立 TCP 连接时刻起，到收到全部内容，所需的时间至少是（　）。

A. 30 ms　　B. 40 ms　　C. 50 ms　　D. 60 ms

解析。原题答案采用教材模型：慢开始初始窗口为 1 MSS，使用同一条持久连接，HTML 收到后才请求图像。TCP 建立用 1RTT；HTML 请求与 1 MSS 响应再 1RTT，服务器收到其确认后窗口可增长为 2 MSS；图像有 3 MSS，先发 2 MSS，收到确认后再发余下 1 MSS，图片阶段还需 2RTT。因此共 4RTT=40 ms，选 **B**。

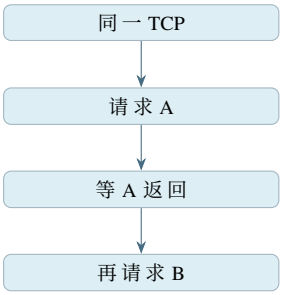
这里题源未在题干单独写初始拥塞窗口，不能把结论推广为所有 HTTP/1.1 实现的固定最短时间。若明确允许较大初始拥塞窗口，3 MSS 图片可在一轮内返回，则题目的数值会改变。遇到这类题，应在计算旁写清采用的 TCP 初态，不能一边沿用 1 MSS 答案、一边声称初态完全无关。

HTTP 缓存还要区分新鲜命中和条件验证。新鲜缓存可直接响应而不向源站发送请求；过期或需重新验证时，可用 If-Modified-Since 或 If-None-Match 等条件请求，服务器返回 304 时不传原完整实体，但验证过程仍有请求响应时延。代理缓存能缩短远程路径、减少上游流量，但并不意味着缓存中的对象永不改变。

回看各应用协议，可按“谁向谁做什么、用何种传输、端口属于哪一方”比较，不能只把名称和端口罗列成表，就以为各协议可以互相替代。

最小反例·持久连接仍可逐个等待

已建立一条 TCP 连接，先请求图片 A，等它完整返回后才请求 B，这是持久但非流水。两对象仍付两次请求响应等待，所以“用了持久连接就所有对象只要 1 RTT”的说法不成立。



持久连接仍可以不使用流水

应用任务与协议	主要交互端点和动作	本章基本承载与端口	需要保留的条件
名称解析 DNS	主机/解析器向 DNS 服务器询问记录	UDP 或 TCP，服务端 53	缓存、递归/迭代决定查询路径；不是每次必经根
自动配置 DHCP	客户向服务器申请地址与其他配置	UDP，服务器 67、客户 68	初始广播与续租单播要按阶段判断
文件传送 FTP	客户和服务器先交换控制命令，再传数据	TCP 控制连接服务端 21；数据另建连接	主动/被动决定数据连接发起方和数据端口
发送邮件 SMTP	用户代理提交、邮件服务器之间转送	TCP，服务器间基本端口 25	推送邮件，不承担用户读取邮箱的角色
读取邮件 POP3	用户代理向邮件服务器读取	TCP，基本服务端 110	一条连接可处理多封邮件，读取不必立即删除
获取 Web 资源 HTTP	浏览器向 Web 服务器请求资源	本章 HTTP/1.x 采用 TCP，默认服务端 80	连接复用、流水、缓存和资源依赖分别判断

这些是本章基础模型的端口与承载，显式 URL 端口、加密服务或其他扩展须按题意另算。MIME 说明邮件内容类型与编码，不取代 SMTP 的投递过程；HTTP 的无状态也不取代 TCP 连接管理。遇到跨协议题，先列应用依赖，再把 DNS 查询、连接建立、请求响应与数据发送逐项接入时间线。

6.9 综合依赖链：名称、连接与逐跳封装

跨章节题先列必须完成的应用动作，再把每个动作接到传输连接和本地链路上。DNS 负责名称，ARP 负责当前邻居 MAC，TCP 负责相应连接的序号与窗口；交换机只从已经实际到达的帧学习，不会提前知道未来哪台主机将参与通信。

不变量·应用数据量与 FIN 分开计

FTP 文件 18000 B，从 SYN 序号 100 开始，则首数据 101，末数据 18100，FIN 为 18101，对方确认 FIN 为 18102。控制连接已有自己的序号空间，不能混进这条数据连接的累计字节数。

6.9.1 例 6-8　FTP 上传时，把连接、字节号与拥塞轮次放到同一时间轴

例 6.9.1 (配套例题)

配套例题。某网络拓扑如下图。H 登录 FTP 服务器后，向服务器上传 18000 B 文件 F。H 为传输 F 建立数据连接时选择初始序号 100，MSS=1000 B，拥塞控制初始阈值 4 MSS，RTT=10 ms，忽略 TCP 段的传输时延。在 F 的传输过程中，H 均按 MSS 大小发送数据，未发生差错、丢包和乱序。



Figure 6.4: H 经交换机、路由器访问 Internet 中的 FTP 服务器，控制连接与数据连接均由 TCP 承载

（1）FTP 控制连接是持久的还是非持久的？数据连接是持久的还是非持久的？H 登录服务器时建立的 TCP 连接是控制连接还是数据连接？

（2）H 通过数据连接发送 F 时，F 第 1 个字节的序号是多少？断开数据连接过程中，FTP 服务器发送的第二次挥手 ACK 段的确认号是多少？

（3）当 H 收到确认号 2101 的确认段时，拥塞窗口调整为多少？收到确认号 7101 的确认段时，拥塞窗口调整为多少？

（4）H 从请求建立数据连接开始，到确认 F 已被服务器全部接收，至少需要多久？期间应用层数据的平均发送速率是多少？

解析。先把 FTP 会话与一次文件传输分开。控制连接在整个 FTP 会话期间保持，承载登录、命令与控制信息，因此是**持久连接**；本题的数据连接服务于本次文件传输，传完后关闭，是**非持久连接**。登录时先建立的是**控制连接**，已经完成的登录过程不再计入第（4）问的数据连接计时。

数据连接的 SYN 占序号 100，F 首字节从 **101** 开始。18000 B 文件占序号 101～18100，发送完后 H 的 FIN 占 18101，服务器第二次挥手的 ACK 应确认下一个期望序号，故为 **18102**。这里确认的是文件数据再加 FIN，不能把登录控制连接的序号混进来。

拥塞控制沿用教材的轮次模型：初始 $cwnd = 1$ MSS，达到 4 MSS 门限后按每 RTT 增加 1 MSS。先看 ACK 落在哪个时刻：确认号 2101 累计确认 $2101 - 101 = 2000$ B，即前 2 个段。第 1 个 ACK 使 cwnd 从 1 增为 2 MSS；第 2 个 ACK 是第二轮中较早的那个，此时仍在慢开始，cwnd 从 2 增为 **3 MSS**。不能把 “第二轮结束时 4 MSS” 提前用于这个 ACK。

确认号 7101 累计确认 $7101-101=7000$ B，即 $1+2+4=7$ 个段，恰好完成第 3 轮。此轮开始时已到 4 MSS 门限，采用拥塞避免，第 3 轮结束后窗口为 **5 MSS**。下表同时列出数据发送、确认时刻与累计量，时间均从数据连接 SYN 发出算起。

阶段	开始时刻/ms	本轮 cwnd/MSS	本轮发送段数
数据连接握手	0	—	—
数据第 1 轮	10	1	1
数据第 2 轮	20	2	2
数据第 3 轮	30	4	4
数据第 4 轮	40	5	5
数据第 5 轮	50	6	6

阶段	累计发送段数	本轮全部确认到达/ms
数据连接握手	0	10
数据第 1 轮	1	20
数据第 2 轮	3	30
数据第 3 轮	7	40
数据第 4 轮	12	50
数据第 5 轮	18	60

建立数据连接用 1 RTT，18 个段按 1、2、4、5、6 分 5 轮发完，并在最后一轮结束时得到全文件确认，所以总计 6RTT $=$ **60 ms**。期间应用层平均速率为 $18000\times 8/0.060=$ **2.4 Mb/s**，也就是 300000 B/s=0.3 MB/s。分母按题目包含握手、不包含连接释放；不能只用 50 ms 的数据传输阶段，也不能因服务器较早收到最后一比特就省去最后确认的回程。

这里的按轮增长是题目采用的教材近似，阈值前逐 ACK 检查仍有意义；拥塞避免阶段不宜把轮次表误解成每个 ACK 都使 cwnd 增加整整 1 MSS。若要描述某个操作系统的精确逐 ACK 轨迹，还需其拥塞算法和 ACK 策略。

最小反例·缓存命中会减少 ARP 广播

若本次已缓存 Web 服务器 IP，就可直接解析网关 MAC，不必先访问本地 DNS。即使 ARP 表和交换表为空，也不足以证明必出现两次 ARP 广播；该结论还需要一次新的 DNS 查询这一条件。

6.9.2 例 6-9　从 DNS 到 HTTP，交换机只学习实际发送者

例 6.9.2 (配套例题)

配套例题。某网络拓扑如下图，以太网交换机 S 通过路由器 R 与 Internet 互联。路由器接口、本地 DNS、H1、H2 的 IP 地址与 MAC 地址如图。t_0 时 H1 的 ARP 表和 S 的交换表均为空；H1 此时用浏览器通过域名 www.abc.com 请求访问 Web 服务器。$t_1>t_0$ 时，S 第一次收到封装 HTTP 请求报文的以太网帧。其间没有与此次 Web 访问无关的网络通信。

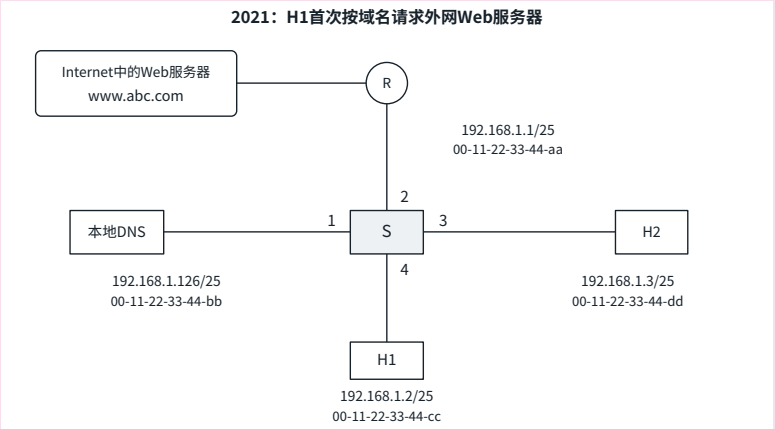


Figure 6.5: 本地 DNS 接 S 的 1 口，R 接 2 口，H2 接 3 口，H1 接 4 口；四者同属 192.168.1.0/25

（1）从 t_0 到 t_1，H1 除 HTTP 外还运行哪个应用层协议？从应用层到数据链路层，该协议报文经过哪些协议逐层封装?

（2）S 交换表结构为 “MAC 地址、端口”，t_1 时交换表内容是什么?

（3）其间 H2 至少接收多少个与此次 Web 访问有关的帧？是什么帧？目的 MAC 地址是什么?

解析。解题入口是按依赖关系列出 “先知道谁的地址，才能发下一帧”。按本题及教材需要新做域名解析的流程，H1 先运行 **DNS**，把域名换成 Web 服务器 IP；常规查询的封装为 **DNS 报文 →UDP 数据报 →IPv4 分组 →Ethernet II 帧**。ARP 用于在本地链路上解析 MAC，不是 DNS 外面的一层，也不是另一个应用层协议；CSMA/CD 是介质访问算法，不是帧格式，在全双工交换链路中通常不运行。

H1 先根据/25 判断本地 DNS 的 192.168.1.126 与自己同属 192.168.1.0/25，先用 ARP 解析本地 DNS 的 MAC 地址，不先问网关。解析出 Web 服务器位于外网后，才 ARP 查询网关 R 的 192.168.1.1。后续 IP 分组的最终目的仍是 Web 服务器，当前以太网帧的目的 MAC 却是 R 的 MAC。

顺序	S 收到的帧	入端口	S 的处理与新增学习
1	H1 广播 ARP，询问本地 DNS 的 MAC	4	学习 H1 的 cc→4；向 1、2、3 口泛洪
2	DNS 单播 ARP 应答给 H1	1	学习 DNS 的 bb→1；已知 cc，只转 4 口
3	H1 单播 DNS 查询给 DNS	4	已知 bb，只转 1 口
4	DNS 单播查询应答给 H1	1	已知 cc，只转 4 口
5	H1 广播 ARP，询问网关 R 的 MAC	4	向 1、2、3 口泛洪
6	R 单播 ARP 应答给 H1	2	学习 R 的 aa→2；只转 4 口
7	H1 向外网服务器发送 TCP SYN，帧发给 R	4	已知 aa，只转 2 口

不变量·应用数据量与 FIN 分开计

FTP 文件 18000 B，从 SYN 序号 100 开始，则首数据 101，末数据 18100，FIN 为 18101，对方确认 FIN 为 18102。控制连接已有自己的序号空间，不能混进这条数据连接的累计字节数。

顺序	S 收到的帧	入端口	S 的处理与新增学习
8	R 转发服务器的 SYN+ACK，帧发给 H1	2	已知 cc，只转 4 口
9	H1 发送 TCP 确认及首个 HTTP 请求	4	帧均发给 R，只转 2 口；到达题设 t_1

课程连接·应用消息和 TCP 段

TCP 按字节流分段，一条应用消息可以跨多个 TCP 段。

表中 cc、bb、aa 分别是下表完整 MAC 地址的末字节缩写。第三次握手 ACK 是否与首个 HTTP 请求合并，不改变学习或 H2 收到的帧数。交换机按**帧的源 MAC 与入端口**学习，不会从广播帧的目的地址、ARP 询问目标、IP 目的地址或题图上 “知道有 H2” 便自行添加表项。因此 t_1 时为：

MAC 地址	端口
00-11-22-33-44-cc（H1）	4
00-11-22-33-44-bb（本地 DNS）	1
00-11-22-33-44-aa（R）	2

H2 只被动收到两次广播 ARP 请求，没有发出自己的帧，故没有 dd→3 条目。两次单播应答到来时，交换机早已从相应请求学到 H1 的位置，不会因初始表为空便把所有单播一直泛洪。第（3）问按上述解析流程，H2 至少收到 **2 个 ARP 请求帧**，分别查询本地 DNS 和网关，目的 MAC 均为 **FF-FF-FF-FF-FF-FF**。

这个 2 帧结论以本题意图中的新 DNS 查询为前提。原题明确为空的是 ARP 表和交换表，并未额外声明 DNS 缓存为空；若允许命中已有 DNS 缓存，就不必先访问本地 DNS，相应学习表与最小广播数都会改变。应保留这一条件边界，不能把 “通过域名访问” 推广成任何情况下都必然重新发送 DNS 查询。

6.10 应用模型与协议服务依赖

C/S 模型由长期运行的服务器等待请求，客户可随时加入或离开。服务器集中管理资源，便于维护和访问控制，也可能形成性能和可用性瓶颈；增加副本和负载均衡是常见扩展。P2P 对等方既获取也提供资源，节点加入会同时带来负载和服务能力；节点可动态变化，定位、访问控制、内容一致性和拥塞仍需协议处理。

应用使用下层服务时，应先问所需的可靠性、时延、吞吐和通信模式，再选择承载。DNS 一般小查询常用 UDP，较大响应、区域传送等可用 TCP；FTP、传统 SMTP/POP3 和本章 HTTP/1.x 使用 TCP。应用层头部与正文也可以被分段，某一个 TCP 段并不天然等于一条完整应用消息。

6.11 DNS 记录与缓存的边界

DNS 资源记录除名称、类型和值之外，还含生存时间 TTL，用于限制缓存使用期限。A 记录给 IPv4 地址，AAAA 给 IPv6 地址，NS 指示权威服务器，MX 指定邮件交换服务器，CNAME 给出别名指向的规范名称。NS 和 MX 中的名称可能还需要解析地址；有了域名不代表已经知道最终 TCP 目的地址。

权威回答负责其管理范围内的数据，本地缓存可以提供尚有效的既有结果。命中目标记录时查询次数可为 0；仅命中某个权威服务器地

址时，可跳过根和顶级步骤，直接问该服务器；完全无相关缓存时按题设从根开始。迭代回答可能给出更接近目标的服务器，而不直接给目标主机地址，因此 “每次收到回答” 与 “域名解析结束” 不同。

辨析·信封与首部

MAIL FROM/RCPT TO 是投递信封，From/To 是邮件内容首部。

6.12 邮件的信封、内容与交互命令

邮件系统由用户代理、邮件服务器和投递/读取协议配合。用户代理负责编辑与呈现，发送服务器将待投递邮件排队，服务器之间由 SMTP 转送；收件用户再通过 POP3、IMAP 或 Web 接口访问邮箱。邮件地址形如**用户标识 @ 域名**，域名部分用于查找邮件服务，不等于邮件服务器 IP 地址的文本形式。

SMTP 基本交互包括建立 TCP 连接、问候、指定发件人、指定收件人、传送邮件内容、结束会话。`MAIL FROM`、`RCPT TO` 属于投递信封，邮件内容中的 `From:`、`To:`、`Subject:`属于邮件首部；多个收件人或转发条件下二者不一定一一相同。DATA 阶段结束标志按 SMTP 协议解释，邮件正文中出现相似内容时由协议透明传输规则处理。

传统 SMTP 以 7 bit ASCII 为基本传输背景。MIME 通过内容类型、传输编码、分段边界等描述非 ASCII 文本、图片和附件，让这些内容可通过相应邮件传送系统。Base64 是编码而不是加密，内容可以被公开解码；MIME 也不代替 SMTP 的实际投递。实际扩展 SMTP 可协商其他能力，应按题目给定的基础模型或扩展条件作答。

POP3 的一次 TCP 连接可处理多封邮件，经历鉴别、事务处理、更新等阶段。下载后删除与下载并保留是不同使用方式；用户端读取不等于 SMTP 服务器间重新投递。Web 邮箱中，浏览器与 Web 服务之间使用 HTTP 类交互，后端仍可能运行 SMTP 等邮件协议，不能根据浏览器界面就把整个邮件系统都归为 HTTP。

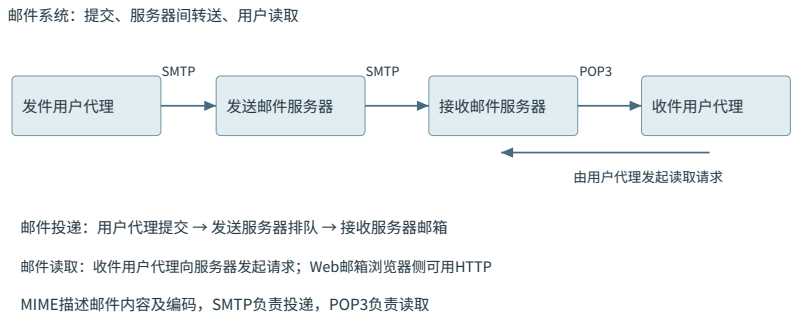


Figure 6.6: 邮件系统的提交、转送和读取路径

6.13 HTTP 报文、Cookie 与缓存验证

HTTP/1.x 请求包含请求行、首部行、空行与可选消息体；响应包含状态行、首部行、空行与可选消息体。请求行给出方法、请求目标与版本，响应状态行给版本、状态码和短语。GET 通常请求资源，HEAD 请求相应首部信息，POST 提交数据；具体语义以方法和资源接口为准。

```
GET /notes/network.html HTTP/1.1
Host: example.test
If-None-Match: "v2"
```

若缓存实体仍有效，服务器可返回 304，验证响应仍消耗一次请求/响应；若实体已变化，通常返回新的完整实体。200 表示成功，301/302 表示重定向类别，404 表示未找到相应资源，5xx 表示服务器错误类别；状态码本身不说明整条 TCP 连接是否已经关闭。

辨析·两种 Cookie 首部

Set-Cookie 来自响应，Cookie 随适用的后续请求发回。

服务器用 `Set-Cookie` 响应首部设置 Cookie，浏览器在后续适用请求中用 `Cookie` 首部带回相应信息。服务器可据标识找到会话状态；HTTP 的每次请求语义与 TCP 连接的持久性分别判断。Cookie 不能自动等同于安全认证，加密、访问范围和会话保护还取决于实际设置。

FTP 也要按消息内容区分连接：`LIST` 命令经控制连接提交，返回的目录列表经数据连接传输。主动模式由服务器发起数据连接，被动模式由服务器先通告监听端点、客户发起连接；控制连接独立维持，所以文件数据流不会把控制命令排在同一字节流后面。

6.14 主动回忆自测

以下为自拟题。时间题先画依赖链，协议题先说谁向谁执行什么动作。

1. HTML 引用 4 个同站小图片，DNS 已完成、TCP 未建立，忽略发送和其他开销。持久非流水与持久流水各需多少 RTT?
2. 本地 DNS 已有目标域权威服务器地址、没有目标主机地址；到权威服务器 RTT20 ms，TCP 建立和 HTTP 请求响应各 30 ms。局域网开销忽略，取纯文本页面至少多久?
3. 收到 304 响应，能否说明本次完全没有网络往返?
4. C/S 中的 “客户” 是否等于性能较低的机器? P2P 使用集中索引后是否就只能由中心转发全部文件数据?
5. 域与区为什么不必相同? A、AAAA、NS、MX 各表达什么?
6. 一次 FTP 登录与后续文件上传分别用哪类连接? 所有 FTP 服务器数据端口都固定 20 吗?
7. SMTP、POP3 与 MIME 分别做什么? Base64 编码是否提供保密性?
8. URL 中的片段 `#part1` 是否通常随 HTTP 请求目标发给服务器? HTTP 无状态能否推出不能使用持久 TCP 连接?
9. 在新做 DNS 解析、ARP 表和交换表为空的局域网访问流程中，交换机从目的 MAC 还是源 MAC 学习端口? DNS 缓存命中会改变哪些步骤?

6.14.1 核对答案

1. 非流水 $1+1+4=6$ RTT，流水 $1+1+1=3$ RTT；两者均包括首次 TCP 握手。 **2.** DNS20 ms，加 Web 两次 30 ms，共 80 ms；无需重新查询根与顶级服务器。 **3.** 不能，304 是网络条件验证响应，只省略完整实体传输。

4. 客户主动请求服务；集中索引可仅协助发现，文件仍可在对等节点间传输。 **5.** 子域可委派给不同权威管理范围；四类记录依次指 IPv4、IPv6 地址、权威服务器和邮件交换。 **6.** 登录走控制连接，文件走数据连接；被动模式的数据端口由服务器通告。

7. SMTP 提交/转送、POP3 读取、MIME 描述内容类型与编码；Base64 可公开解码，不是加密。 **8.** 片段留在浏览器侧；无状态与连接复用是不同层次的问题，可以使用持久连接。 **9.** 从源 MAC 和入端口学习；DNS 命中可省去查询及其 ARP，改变交换表和广播计数。